Compilers Theory
3CS – CS309-Credit

Lexical Analysis

Dr. Lamiaa Hassaan

What is Lexical Analysis Phase?

- It is the first phase of the compiler.
- It reads the input characters and produces as output a sequence of tokens that the parser uses for syntax analysis.
- It also removes comments and white spaces in the form of blank , tab , and newline characters from the source program.
- It also correlates error messages from the compiler with the source program.

Alphabets and Strings

- **An alphabet (Σ)** is a finite nonempty ordered set of symbols, letters or characters.
- **A string:** which is also called a ***word*** or ***token*** is a finite sequence of symbols from a given alphabet.
- **e.**g. 011 is a string of length three over alphabet **Σ** = {0, 1}
- 0101 is a string of length four over alphabet **Σ** = {0, 1}
- *for* is a string of length three over the alphabet of C++ language.

Operations Performed on a Language

Union: the union of two sets is the set with contains all the elements in each of the two sets with nothing else. The union of two languages L1={abc, ab, ba} and L2={ba, bb} is {abc, ab, ba, bb}.
Notice that L1UL2 = L2UL1

Concatenation: the concatenation of two languages is the language formed by concatenating each string in one language with each string in the other language. Concatenating the languages {ab, c} and { d, ef } produces {abd, abef, cd, cef}.

Operations Performed on a Language

- Concatenation ‘continued’

Assume the two languages L1 = {a,b} and L2 = {0,11}

Then,

L1L2 = {a0, a11, b0,b11}

L2L1= {0a, 0b, 11a,11b}

Notice that L1L2 ≠ L2L1

Operations Performed on a Language

- Kleene closure: Σ^* is the set of all strings that can be made by concatenating zero or more strings in Σ. $\{a\}^* = \{\varepsilon, a, aa, aaa, aaaa, \ldots\}$.
- What is ε transition? It is a notation of an empty string, such as Φ, so that $\{\varepsilon\}$ is a set of one element which is the empty string.
- Positive closure: Σ^+ is the set of all strings that can be made by concatenating one or more strings in Σ. $\{a\}^+ = \{a, aa, aaa, aaaa, \ldots\}$.
- The **or** operator | $a|b|c=\{a,b,c\}$

Regular Expressions

- The set of all integer constants or the set of all variable names are sets of strings, where the individual letters are taken from a particular alphabet.
- Such a set of strings is called a language. For integers, the alphabet consists of the digits 0-9 and for variable names the alphabet contains both letters and digits (and perhaps a few other characters, such as underscore).
- Example: Consider the alphabet X={a,b,c}, the following are considered to be regular expressions on the language X: ab*, a|b|c*.....

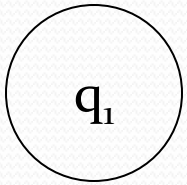
Algebraic Properties of Regular Expressions

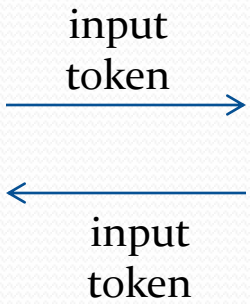
$(r\|s)\|t = r\|s\|t = r\|(s\|t)$	$\|$ is associative
$s\|t = t\|s$	$\|$ is commutative
$s\|s = s$	$\|$ is idempotent
$s? = s\|\varepsilon$	by definition
$(rs)t = rst = r(st)$	concatenation is associative
$s\varepsilon = s = \varepsilon s$	ε is a neutral element for concatenation
$r(s\|t) = rs\|rt$	concatenation distributes over $\|$
$(r\|s)t = rt\|st$	concatenation distributes over $\|$
$(s^*)^* = s^*$	* is idempotent
$s^*s^* = s^*$	0 or more twice is still 0 or more
$ss^* = s^+ = s^*s$	by definition

Table : Algebric properties of regular expressions

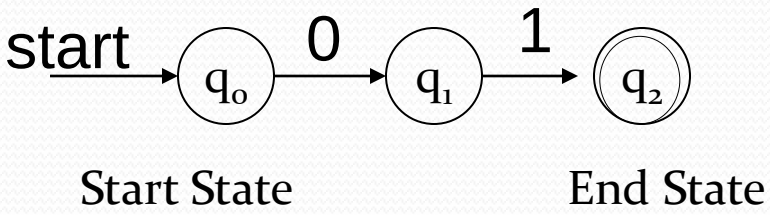
Finite Automata

- Finite automata are used to recognize the tokens specified by a regular expression and can be converted to an algorithm for matching input strings.
- A finite automaton is, in the abstract sense, a machine that has a finite number of ***states*** and a finite number of ***transitions*** between these.

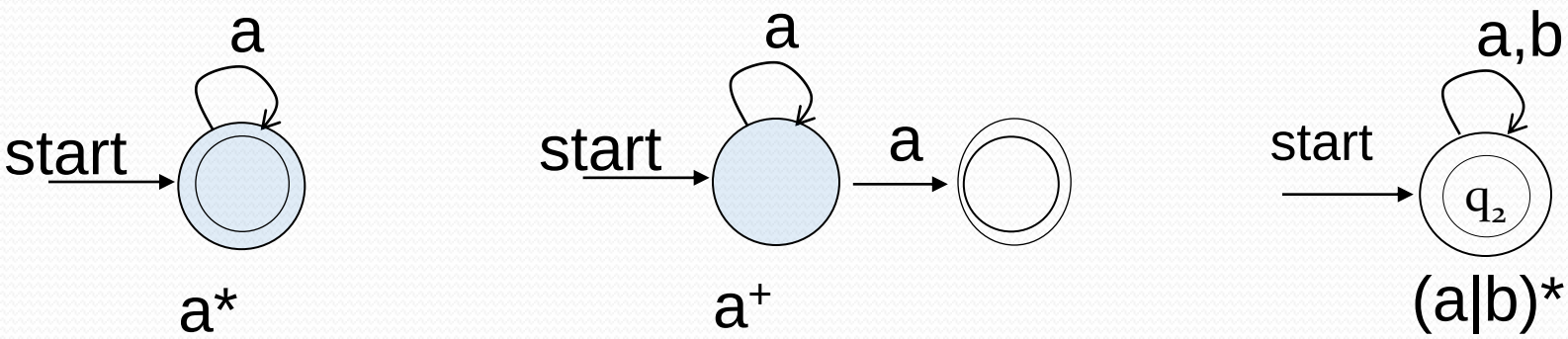
State 

Transition 

Finite Automata



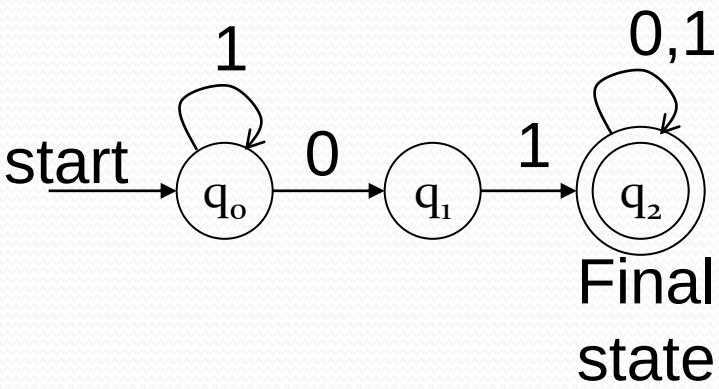
A finite automata for the expression 01

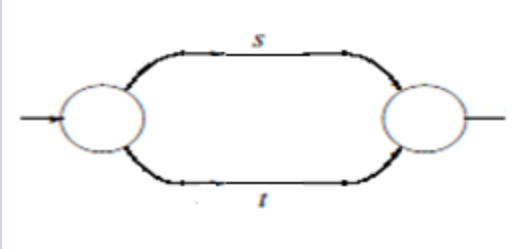
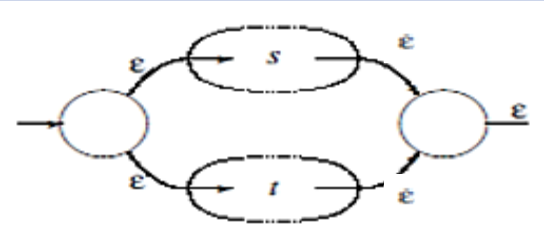


Finite Automata

- Construct a finite automaton to recognize the following regular expression, :

1*01(0|1)*



NFA vs. DFA

	DFA	NFA
Empty string transition	DFA is not capable of using an Empty String transition.	NFA can use an empty String transition.
Transition functions	The number related to the next state is one.	The number related to the next state/ states is either zero or one.
Complexities of time	The total time required for running any input string in DFA is less than what it is in NFA.	The total time required for running any input string in NFA is larger than that in comparison to DFA.
The setting of the next possible set	The next possible state is clearly set in DFA.	Every single pair of input symbols and states may contain many possible next states.
Example